

Docker FAQ for Beginners

To Begin with the Lab

Summary of the Lab

In this lesson, Docker was explained through practical beginner questions. The video showed how a Docker host can be created by installing Docker on an EC2 instance, how containers differ from virtual machines, and where container images come from. It also demonstrated that containers can be started, stopped, and restarted quickly, how to create a custom Docker image using a Dockerfile, how to expose a container to the outside world using port mapping, and why a single Docker host can become a single point of failure. The lesson ended by pointing toward orchestration tools such as ECS, Kubernetes, and Docker Swarm for high availability.

Understanding Docker Containers

Docker is a container platform that runs applications in isolated environments. Unlike virtual machines, containers do not include their own kernel. Instead, they share the host's kernel, which is why they are lightweight and start very quickly. In this lesson, the instructor used a Linux EC2 instance as a Docker host and explained that Docker images can be pulled from Docker Hub, then used to create containers in about one second.

Lab Steps

Step 1 – Create a Docker Host

- Sign in to the AWS Management Console.
- Open **Amazon EC2**.
- Launch a Linux instance such as Amazon Linux with name as **Docker-Host**.

Name
Docker-Host

Application and OS Images (Amazon Machine Image)
An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recents | My AMIs | **Quick Start**

Amazon Linux | macOS | Ubuntu | Windows | Red Hat | SUSE Linux | Debian

Amazon Machine Image (AMI)
Amazon Linux 2023 kernel-6.18 AMI
ami-06067086cf86c58e6 (64-bit (x86), uefi-preferred) / ami-071bc399fae13263b (64-bit (Arm), uefi)
Virtualization: hvm | ENA enabled: true | Root device type: ebs

Description
Amazon Linux 2023 (kernel-6.18) is a modern, general purpose Linux-based OS that will be supported until June 2029. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.12.20260629.0 x86_64 HVM kernel-6.18

Summary
Number of instances | Info
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.12...[read more](#)
ami-06067086cf86c58e6

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as

Cancel | **Launch instance** | [Preview code](#)

- Allow HTTP traffic and select proper Key pair.

Key pair name - required
June-2026-us

Network settings
Network | Info
vpc-0...

Subnet
No preference (Default subnet in any availability zone)

Auto-assign public IP
Enable
Additional charges apply when outside of free tier allowance

Firewall (security groups)
Create security group | Select existing security group

We'll create a new security group called 'launch-wizard-11' with the following rules:

☒ Allow SSH traffic from
Helps you connect to your instance
Anywhere
0.0.0.0/0

☐ Allow HTTPS traffic from the internet
To set up an endpoint, for example when creating a web server

☒ **Allow HTTP traffic from the internet**
To set up an endpoint, for example when creating a web server

Summary
Number of instances | Info
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.12...[read more](#)
ami-06067086cf86c58e6

Virtual server type (instance type)
t3.micro

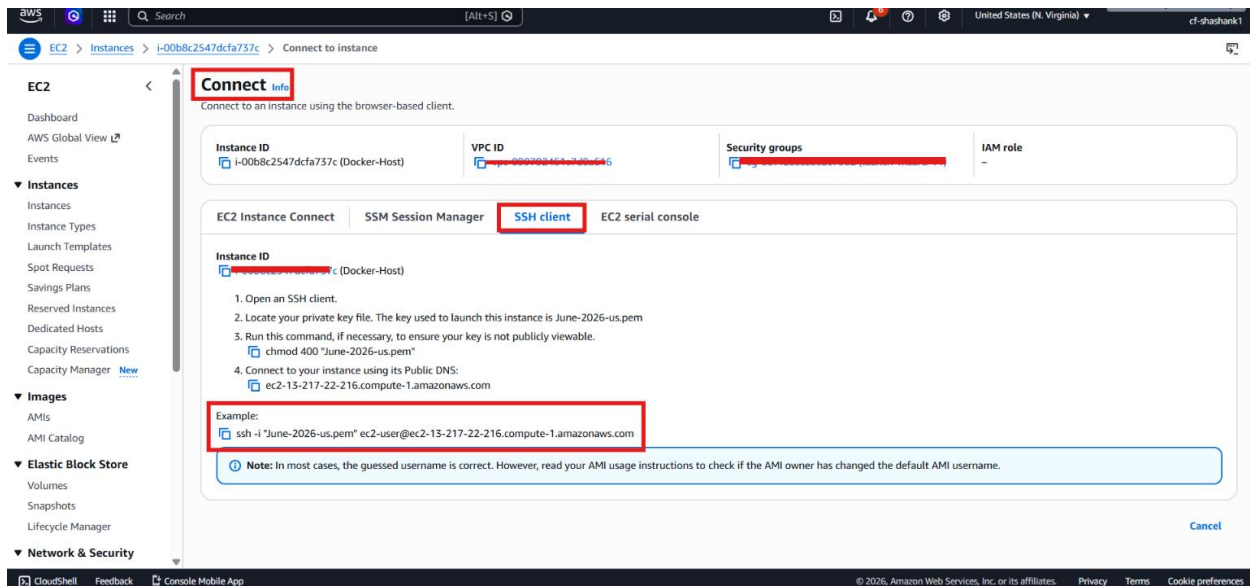
Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as

Cancel | **Launch instance** | [Preview code](#)

- Connect to the instance using SSH command as shown and run it in Command Prompt.

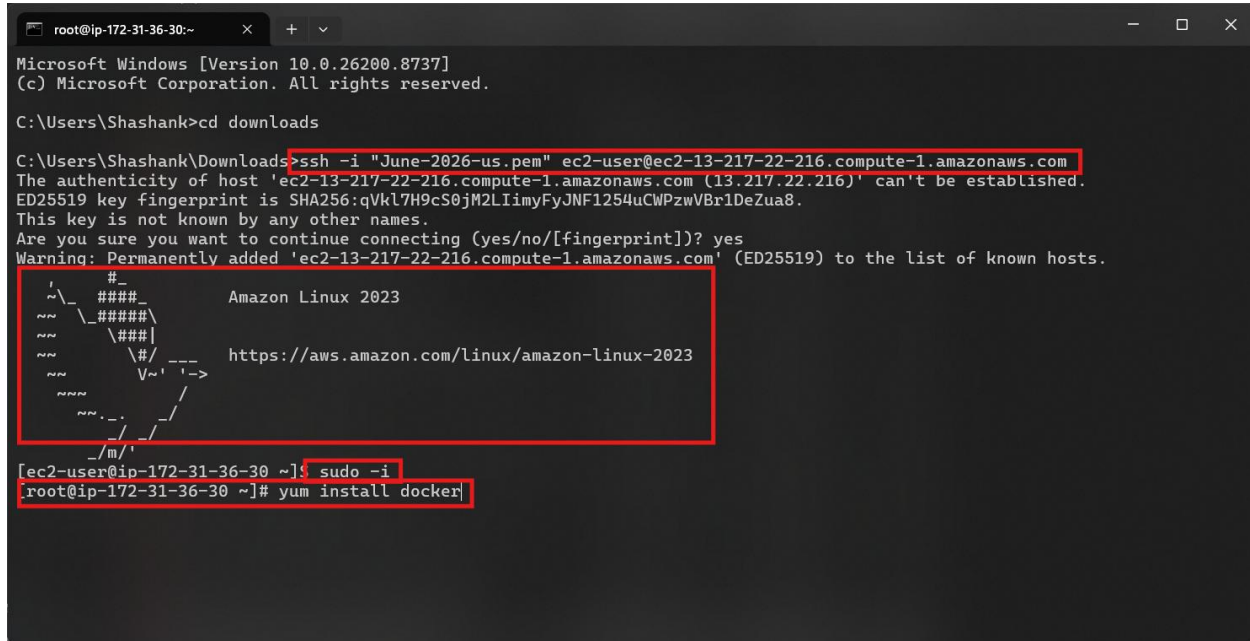


- Install Docker daemon on the instance and for that login as root user via this command.

```
sudo -i
```

- Docker can be installed using this command.

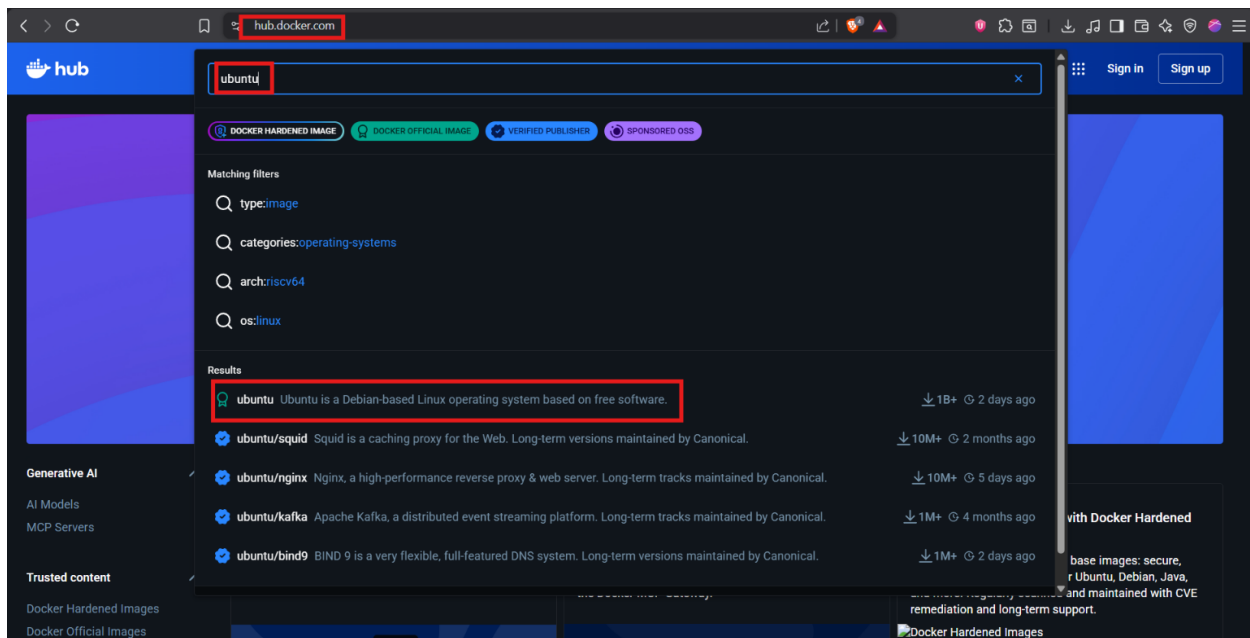
```
yum install docker
```



- After installation, the EC2 instance becomes a Docker host.

Step 2 – Understand Container Images

- Open **Docker Hub**.
- hub.docker.com
- Search for an image such as:
 - ubuntu



- Download the required image.
- Understand that container images are not ISO files like virtual machines.
- Remember that the container shares the host kernel, so the image is much smaller than a VM image.

Step 3 – Create and Run a Container

- Start the docker using this command

```
systemctl start docker
```

- To verify if its working run this command, it should show version of docker.

```
docker --version
```

- Pull the image if needed and its command can be found on the docker hub site.

- Run the command from the site.
- To verify the images, you can run this command.

docker images

```

root@ip-172-31-36-30:~
Verifying      : libcgrou-3.0-1.amzn2023.0.1.x86_64      6/11
Verifying      : libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64      7/11
Verifying      : libnftnl-1.0.1-19.amzn2023.0.2.x86_64      8/11
Verifying      : libnftnl-1.2.2-2.amzn2023.0.2.x86_64      9/11
Verifying      : pigz-2.5-1.amzn2023.0.4.x86_64      10/11
Verifying      : runc-1.3.5-1.amzn2023.0.2.x86_64      11/11

Installed:
container-selinux-4:2.245.0-1.amzn2023.noarch
docker-25.0.16-1.amzn2023.0.2.x86_64
iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
libnftnl-1.2.2-2.amzn2023.0.2.x86_64
runc-1.3.5-1.amzn2023.0.2.x86_64
containerd-2.2.4-1.amzn2023.0.3.x86_64
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
libcgrou-3.0-1.amzn2023.0.1.x86_64
libnftnl-1.0.1-19.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.4.x86_64

Complete!
[root@ip-172-31-36-30 ~]# systemctl start docker
[root@ip-172-31-36-30 ~]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-36-30 ~]# docker pull ubuntu:stonking-20260622
stonking-20260622: Pulling from library/ubuntu
3b86f06e/cb7: Pull complete
0fd7b49bedf6: Pull complete
Digest: sha256:68b4e16d3b586c94c232ca319a42d0a48d3d778553cbe5a07c854d0ea5eecd69
Status: Downloaded newer image for ubuntu:stonking-20260622
docker.io/library/ubuntu:stonking-20260622
[root@ip-172-31-36-30 ~]# docker images
REPOSITORY    TAG                IMAGE ID          CREATED          SIZE
ubuntu        stonking-20260622  2e7ea2d9ce82     12 days ago     100MB

```

- Create a container from the image.

docker run -it --name c1 ubuntu /bin/bash

```
root@85a06e23421b: /
Installed:
container-selinux-4:2.245.0-1.amzn2023.noarch
docker-25.0.16-1.amzn2023.0.2.x86_64
iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
libnftnl-1.2.2-2.amzn2023.0.2.x86_64
runc-1.3.5-1.amzn2023.0.2.x86_64
containerd-2.2.4-1.amzn2023.0.3.x86_64
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
libcgroup-3.0-1.amzn2023.0.1.x86_64
libnftnl-1.0.1-19.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.4.x86_64

Complete!
[root@ip-172-31-36-30 ~]# systemctl start docker
[root@ip-172-31-36-30 ~]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-36-30 ~]# docker pull ubuntu:stonking-20260622
stonking-20260622: Pulling from library/ubuntu
3b86f66e7c67: Pull complete
0fd7b49bedf6: Pull complete
Digest: sha256:68b4e16d3b586c94c232ca319a42d0a48d3d778553cbe5a07c854d0ea5eecd69
Status: Downloaded newer image for ubuntu:stonking-20260622
docker.io/library/ubuntu:stonking-20260622
[root@ip-172-31-36-30 ~]# docker images
REPOSITORY    TAG                IMAGE ID           CREATED          SIZE
ubuntu        stonking-20260622  2e7ea2d9ce82      12 days ago     100MB
[root@ip-172-31-36-30 ~]# docker run -it --name c1 ubuntu /bin/bash
unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
a9be9fd915e9: Pull complete
2c1ce1d0a589: Pull complete
Digest: sha256:b7f48194d4d8b763a478a621cdc81c27be222ba2206ca3ca6bc42b49685f3d9e
Status: Downloaded newer image for ubuntu:latest
root@85a06e23421b: /#
```

- Observe that the container starts very quickly.
- Exit the container when finished.
- Use Ctrl + P then Ctrl + Q to detach without stopping the container.

```
root@ip-172-31-36-30: ~
Installed:
container-selinux-4:2.245.0-1.amzn2023.noarch
docker-25.0.16-1.amzn2023.0.2.x86_64
iptables-nft-1.8.8-3.amzn2023.0.2.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64
libnftnl-1.2.2-2.amzn2023.0.2.x86_64
runc-1.3.5-1.amzn2023.0.2.x86_64
containerd-2.2.4-1.amzn2023.0.3.x86_64
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64
libcgroup-3.0-1.amzn2023.0.1.x86_64
libnftnl-1.0.1-19.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.4.x86_64

Complete!
[root@ip-172-31-36-30 ~]# systemctl start docker
[root@ip-172-31-36-30 ~]# docker --version
Docker version 25.0.14, build 0bab007
[root@ip-172-31-36-30 ~]# docker pull ubuntu:stonking-20260622
stonking-20260622: Pulling from library/ubuntu
3b86f66e7c67: Pull complete
0fd7b49bedf6: Pull complete
Digest: sha256:68b4e16d3b586c94c232ca319a42d0a48d3d778553cbe5a07c854d0ea5eecd69
Status: Downloaded newer image for ubuntu:stonking-20260622
docker.io/library/ubuntu:stonking-20260622
[root@ip-172-31-36-30 ~]# docker images
REPOSITORY    TAG                IMAGE ID           CREATED          SIZE
ubuntu        stonking-20260622  2e7ea2d9ce82      12 days ago     100MB
[root@ip-172-31-36-30 ~]# docker run -it --name c1 ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
a9be9fd915e9: Pull complete
2c1ce1d0a589: Pull complete
Digest: sha256:b7f48194d4d8b763a478a621cdc81c27be222ba2206ca3ca6bc42b49685f3d9e
Status: Downloaded newer image for ubuntu:latest
root@85a06e23421b: /# [root@ip-172-31-36-30 ~]#
```

Step 4 – Manage the Container

- View running containers.

```
docker ps
```

- View all containers, including stopped ones.

```
docker ps -a
```


- Stop a container.

```
docker stop <id>
```

- Start a container again.

```
docker start <id>
```

- Attach back to the running container if needed.

```
docker attach <id>
```

```

root@85a06e23421b: /
3b86f66e7c67: Pull complete
0fd7b49bedf6: Pull complete
Digest: sha256:68b4e16d3b586c94c232ca319a42d0a48d3d778553cbe5a07c854d0ea5eecd69
Status: Downloaded newer image for ubuntu:stonking-20260622
docker.io/library/ubuntu:stonking-20260622
[root@ip-172-31-36-30 ~]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
ubuntu stonking-20260622 2e7ea2d9ce82 12 days ago 100MB
[root@ip-172-31-36-30 ~]# docker run -it --name c1 ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
a9be9fd915e9: Pull complete
2c1ce1d0a589: Pull complete
Digest: sha256:b7f48194d4d8b763a478a621cdc81c27be222ba2206ca3ca6bc42b49685f3d9e
Status: Downloaded newer image for ubuntu:latest
root@85a06e23421b:/# [root@ip-172-31-36-30 ~]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
85a06e23421b ubuntu "/bin/bash" 4 minutes ago Up 4 minutes c1
[root@ip-172-31-36-30 ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
85a06e23421b ubuntu "/bin/bash" 4 minutes ago Up 4 minutes c1
[root@ip-172-31-36-30 ~]# docker stop c1
c1
[root@ip-172-31-36-30 ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
85a06e23421b ubuntu "/bin/bash" 5 minutes ago Exited (137) 17 seconds ago c1
[root@ip-172-31-36-30 ~]# docker start c1
c1
[root@ip-172-31-36-30 ~]# docker attach c1
root@85a06e23421b:/#

```

Step 5 – Create a Custom Image

- Create a working directory for the application.
- So for that we have to come out of container by Ctrl + P then Ctrl + Q.
- Now we will make a directory for the app and switch to that directory.

```
mkdir myapp
```

```
cd myapp
```

- Add your application file, such as index.html and paste it in vi editor.

```
vi index.html
```

```
root@ip-172-31-36-30:~/myapp X + v
Status: Downloaded newer image for ubuntu:stonking-20260622
docker.io/library/ubuntu:stonking-20260622
[root@ip-172-31-36-30 ~]# docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
ubuntu stonking-20260622 2e7ea2d9ce82 12 days ago 100MB
[root@ip-172-31-36-30 ~]# docker run -it --name c1 ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
a9be9fd915e9: Pull complete
2c1ce1d0a589: Pull complete
Digest: sha256:b7f48194d4d8b763a478a621cdc81c27be222ba2206ca3ca6bc42b49685f3d9e
Status: Downloaded newer image for ubuntu:latest
root@85a06e23421b:/# [root@ip-172-31-36-30 ~]# docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
85a06e23421b ubuntu "/bin/bash" 4 minutes ago Up 4 minutes c1
[root@ip-172-31-36-30 ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
85a06e23421b ubuntu "/bin/bash" 4 minutes ago Up 4 minutes c1
[root@ip-172-31-36-30 ~]# docker stop c1
c1
[root@ip-172-31-36-30 ~]# docker ps -a
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
85a06e23421b ubuntu "/bin/bash" 5 minutes ago Exited (137) 17 seconds ago c1
[root@ip-172-31-36-30 ~]# docker start c1
c1
[root@ip-172-31-36-30 ~]# docker attach c1
root@85a06e23421b:/# read escape sequence
[root@ip-172-31-36-30 ~]# mkdir myapp
[root@ip-172-31-36-30 ~]# cd myapp
[root@ip-172-31-36-30 myapp]# vi index.html
```

- Now paste the code provided in Resources folder and exit the editor by pressing w then q, if it don't work then type ZZ (Capital Z two times).

```
root@ip-172-31-36-30:~/myapp X + v
text-decoration: none;
font-weight: 700;
transition: transform .2s ease, box-shadow .2s ease, background .2s ease;
box-shadow: 0 10px 24px rgba(255, 159, 46, 0.28);
}
a: hover {
transform: translateY(-2px);
background: #ffab49;
}
.tag {
display: inline-block;
margin-bottom: 18px;
padding: 8px 14px;
border-radius: 999px;
background: rgba(255, 255, 255, 0.1);
color: rgba(255, 255, 255, 0.9);
font-size: 0.88rem;
}
</style>
</head>
<body>
<div class="card">
<div class="tag">CloudFreeeks</div>
<h1>CloudFreeeks</h1>
<p>Welcome to a modern cloud-powered platform built for learning, building, and deploying with confidence.</p>
<a href="#">Visit CloudFreeeks</a>
</div>
</body>
</html>
```

- Write a Dockerfile.

vi Dockerfile


```
root@ip-172-31-36-30:~/myapp$ docker build -t myapp .
[+] Building 2.1s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 165B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 2.13kB
=> [1/2] FROM docker.io/library/nginx:alpine@sha256:54f2a904c251d5a34adf545a72d32515a15e08418dae0266e23be2e18c66
=> => resolve docker.io/library/nginx:alpine@sha256:54f2a904c251d5a34adf545a72d32515a15e08418dae0266e23be2e18c66
=> => extracting sha256:e6f31ffc071e5560b82a8685fba8214954e5721e3e49269d00958316edbe89fe
=> => sha256:35cd77497979abe70dc8d26f5ae60811eea233a2eb5dc03c2ee30972caeb303e 2.50kB / 2.50kB
=> => sha256:ea51152ef8c480b999cee0271a288c698704b18483276119b1253e576ef3232f 12.32kB / 12.32kB
=> => sha256:e6f31ffc071e5560b82a8685fba8214954e5721e3e49269d00958316edbe89fe 3.84MB / 3.84MB
=> => sha256:c16defe09b2f86b04c4143bb610095be90732794fce5c56fb7185f02feadd879 1.89MB / 1.89MB
=> => sha256:5b429a43b8dfa079c3ac95537bd88d5f1ac70e478c64f100b8ef6aa9c555ebc2 628B / 628B
=> => sha256:54f2a904c251d5a34adf545a72d32515a15e08418dae0266e23be2e18c66fefa 10.33kB / 10.33kB
=> => sha256:967885d218c57d3ce2a4e906131fb25f59e6f56cce51d87dde7d74b0e7465675 955B / 955B
=> => sha256:ce42635eeddd348e8266227a36db92cea8a45d5177d03e9a922a7bfb25762b7f 1.21kB / 1.21kB
=> => sha256:c75b9c33e8b0983941fe0379379bd5d137f641f92b4a94d1c8956d28ba77f65 20.25MB / 20.25MB
=> => sha256:ab1fd90497517c799d4fb351bca7ae8b58e231345d4948ae8ac73c75b320b35 405B / 405B
=> => sha256:01bf363d61e6136ebd7dcaf74b303bd08ee8f849a04e2c0be5a8d03159b404f6 1.40kB / 1.40kB
=> => extracting sha256:c16defe09b2f86b04c4143bb610095be90732794fce5c56fb7185f02feadd879
=> => extracting sha256:5b429a43b8dfa079c3ac95537bd88d5f1ac70e478c64f100b8ef6aa9c555ebc2
=> => extracting sha256:967885d218c57d3ce2a4e906131fb25f59e6f56cce51d87dde7d74b0e7465675
=> => extracting sha256:ab1fd90497517c799d4fb351bca7ae8b58e231345d4948ae8ac73c75b320b35
=> => extracting sha256:ce42635eeddd348e8266227a36db92cea8a45d5177d03e9a922a7bfb25762b7f
=> => extracting sha256:01bf363d61e6136ebd7dcaf74b303bd08ee8f849a04e2c0be5a8d03159b404f6
=> => extracting sha256:c75b9c33e8b0983941fe0379379bd5d137f641f92b4a94d1c8956d28ba77f65
=> [2/2] COPY index.html /usr/share/nginx/html/index.html
=> exporting to image
=> => exporting layers
=> => writing image sha256:7af9a32003646f24da2ca5d8f0fa3130a45cefcf1efa0e362f12f845ce53136f
=> => naming to docker.io/library/myapp
[root@ip-172-31-36-30 myapp]$
```

Step 6 – Expose the Container to the Outside World

- Run the container in detached mode.
- Map port 80 on the host to port 80 in the container.

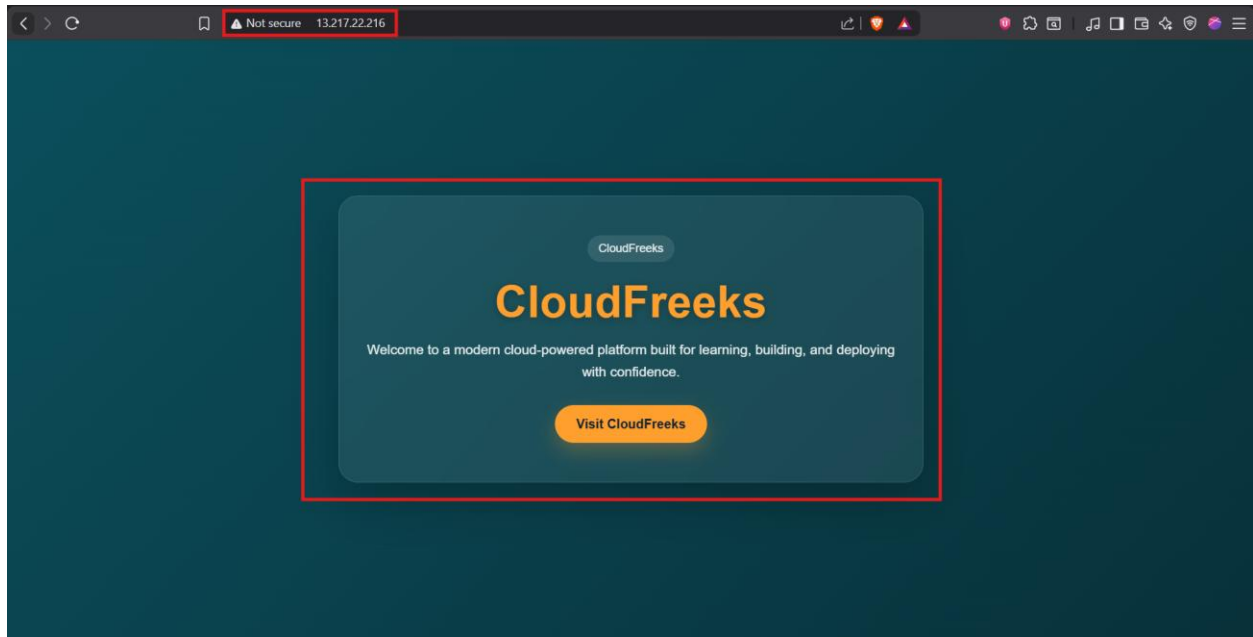
```
docker run -d -p 80:80 myapp
```

```
root@ip-172-31-36-30:~/myapp # docker run -d -p 80:80 myapp
349f7ffc18153db73ff351751a8dcb7685a6611332d4ce4015c7112e3339a33c
root@ip-172-31-36-30 myapp]#
```

- Open the EC2 public IP in a browser.

The screenshot shows the AWS Management Console interface. On the left, the 'EC2' menu is expanded, showing 'Instances'. The main panel displays a list of instances. The first instance, 'Docker-Host' with ID 'i-00b8c2547dca737c', is highlighted. Below the list, the 'Details' tab for this instance is open. A red box highlights the 'Public IPv4 address' field, which shows the IP address '13.217.22.216'. Other details like 'Instance state' (Running) and 'Public DNS' are also visible.

- Confirm that the web application is reachable from outside the container.



Step 7 – Understand Single Host Failure

- Note that if the Docker host fails, all containers on that host fail too.
- Understand that this is a single point of failure.
- Use orchestration tools such as ECS, Kubernetes, or Docker Swarm to improve availability and manage containers across multiple nodes.

Verification

- Docker was installed successfully on the EC2 instance.
- A Docker host was created.
- A container was started from a downloaded image.
- The container could be stopped and started again.
- A custom image was created using a Dockerfile.
- The application was exposed using port mapping.
- The single-host failure problem was understood clearly.